МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ

лабораторної роботи №1

«Розгортання середовища та порівняльний аналіз мовних моделей»

Виконав: студент групи AI-243
Гладковський Іван

Перевірив: ас. Баранов О.М.

Одеса – 2026

Конфігурація середовища

Компонент	Значення
Процесор	AMD Ryzen 7 5800X (8 ядер)
Відеокарта	NVIDIA GeForce RTX 3070
Оперативна пам'ять	32 ГБ
Локальні моделі (Ollama)	qwen3:4b (2,5 ГБ, міркувальна), llama3.2:3b (2,0 ГБ, інструктивна)
Хмарна модель	gemini-3.5-flash-lite (Google AI Studio, OpenAI-сумісний ендпойнт)
Обчислення локальних моделей	100% GPU (за виводом ollama ps)
Клієнт	Python, бібліотека openai (venv), ключ у .env (не потрапляє до репозиторію)

Усі вимірювання виконано на одному комп'ютері, запити надсилались послідовно, без паралельного навантаження. Результати швидкості залежать від цього заліза й не є універсальними. Скрипти та сирі дані лежать поруч зі звітом (providers.py, benchmark.py, quality.py, sampling.py, results.csv, quality.json та допоміжні файли).

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Запити виконано до ендпойнта /api/generate (stream=false) скриптом task1_reasoning.py, який замінює curl. Час = total_duration/1e9, токенів/с = eval_count/(eval_duration/1e9). Сирі дані: task1_raw.json.

Таблиця 1. Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Киев	Київ
eval_count	3217	4
Час, с	29.15	0.05
Токенів/с	110.8	166.8

Таблиця 2. Задача на міркування «ручка і зошит» (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1582.8	220.2
Час, с	13.95	1.69
Токенів/с (середнє)	114.5	133.0

Помилки llama3.2:3b: один прогін відповів «100 грн» (переплутала, що саме дешевше), два прогони дали «50 грн» (замінили умову «на 100 грн дешевша» на «на 10 грн дешевша»). У чотирьох із п'яти прогонів відповідь була (принаймні частково) російською, хоча запит українською. Прогін 1, зарахований як правильний, закінчується спотвореною фразою, хоча ціну ручки (5 грн) названо.

Відповіді від чат-бота (Claude, Anthropic)

1. Куди пішли додаткові токени міркувальної моделі?

Відповідь: Вони пішли в «ланцюжок міркувань» (поле thinking): міркувальна модель спершу генерує чернетку з самоперевірками, і лише потім коротку відповідь. Ці токени теж генеруються й тарифікуються, хоча користувач їх не бачить; тут це 3217 токенів проти 4 для однослівного «Київ». Відповіді навіть не збіглися: qwen3:4b написала «Киев» (російською формою).

2. На яких типах задач міркувальні моделі дають перевагу, а на яких їх використання невиправдане?

Відповідь: Міркувальні моделі виграють на багатокрокових задачах: арифметика, логіка, планування, код із кількома умовами, де проміжні результати треба тримати в тексті. На простих фактичних питаннях, перефразуванні, класифікації та форматуванні вони лише

додають затримку й вартість без приросту якості.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Обидва типи проходять попереднє навчання на великому корпусі та інструктивне донавчання. Міркувальні додатково тренують із підкріпленням (RL) на задачах із перевірюваною відповіддю, заохочуючи довгі ланцюжки міркувань, що ведуть до правильного результату; інструктивні вчать видавати готову відповідь одразу.

4. Чи означає більша кількість згенерованих токенів, що модель «розумніша»?

Відповідь: Ні. Кількість токенів показує обсяг виконаної роботи, а не якість: міркування може бути надлишковим або містити помилки. Тут qwen3:4b із ~1583 токенами розв'язала задачу 5 з 5, але на простому питанні витратила майже на три порядки більше токенів за той самий результат.

5. Як пояснити різницю у швидкості генерації (токенів/с) між qwen3:4b та llama3.2:3b?

Відповідь: Швидкість у токенах за секунду залежить від розміру моделі, архітектури, квантування й заліза, а не від того, міркує модель чи ні. Тут qwen3:4b дає ~114.5 ток/с, а llama3.2:3b ~133.0 ток/с: друга трохи швидша, бо менша (3 проти 4 млрд параметрів). Різниця в загальному часі майже повністю походить від кількості згенерованих токенів.

Висновок за завданням 1

Швидкість генерації (токени/с) і загальний час відповіді — різні величини: перша характеризує залізо й розмір моделі, друга — обсяг роботи, який модель собі призначила. На фактичному питанні режим міркування є чистою перевитратою (3217 проти 4 токенів, 29,2 с проти 0,05 с), до того ж qwen3:4b відповіла «Киев». На задачі про ручку ситуація зворотна: qwen3:4b правильна 5 із 5 за ~14 с, llama3.2:3b правильна лише 2 із 5 за ~1,7 с. Тип моделі слід обирати під тип задачі: міркувальну — для кількох кроків, інструктивну — для простих запитів і там, де критична затримка.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Варіант 7: «Поясни різницю між процесом і потоком.» Скрипт hello_system.py, temperature=0,7. Дані: hello_results.json.

Таблиця 3. Вплив system-повідомлення

Модель	System	Довжина, символів	Мова	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	289	українською (зі спотвореннями)	3	коротко так, але зміст хибний
llama3.2:3b	українською, 2 речення	264	українською	2	так за формою; зміст хибний
qwen3:4b	лаконічний асистент	372	українською	2 + список із 3 пунктів	так (коротко й по суті)
qwen3:4b	українською, 2 речення	156	українською	2	так

Таблиця 4. Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	87	137
llama3.2:3b	українською, 2 речення	54	72	126
qwen3:4b	лаконічний асистент	53	2574	2627
qwen3:4b	українською, 2 речення	62	441	503

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: У режимі «українською, 2 речення» обидві дали рівно два речення. За змістом точніша qwen3:4b (суть загалом правильна), тоді як llama3.2:3b описала процес як «чергування взаємодій між системами» — безглуздо. У режимі «лаконічний асистент» llama дала три речення (з них перше беззмістовне), qwen — два речення й короткий список.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: У видимих відповідях цього завдання — ні, усі чотири українською, хоча в llama3.2:3b є спотворені слова. Але в завданні 1 llama відповідала російською на україномовний запит, а qwen3:4b написала «Киев».

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: У llama3.2:3b з 50 до 54 (+4), у qwen3:4b з 53 до 62 (+9). Нове повідомлення довше за «Ти лаконічний технічний асистент». Різний приріст при однаковому тексті пояснюється різними токенізаторами: вони по-різному розбивають українські слова.

4. Чому в qwen3:4b completion_tokens значно більше, ніж у llama3.2:3b, при однаковій довжині видимої відповіді?

Відповідь: qwen3:4b міркувальна: приховане поле thinking також входить у completion_tokens. За system «лаконічний асистент» вона згенерувала 2574 токени заради 372 символів видимого тексту, а llama3.2:3b — 87 токенів.

Відповіді від чат-бота (Claude, Anthropic), крок 7

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Дотримання інструкцій залежить від якості й обсягу інструктивного донавчання, розміру моделі та мови навчальних даних. Менша модель гірше узагальнює вимоги на кшталт «максимум два речення» й «лише українською», особливо поза англомовними шаблонами. Тому llama3.2:3b формально вклалась у два речення, але зміст вийшов хибним.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: З погляду моделі це різні ролі в шаблоні діалогу з окремими службовими токенами; донавчання привчає надавати system вищий пріоритет. Але це статистична схильність, а не жорстке правило: за конфлікту з сильним патерном або з конкретнішим user-повідомленням модель може слідувати user.

3. Чому міркувальна модель витрачає більше вихідних токенів, навіть коли видима відповідь однакової довжини?

Відповідь: Бо токени поля thinking теж є вихідними й входять у completion_tokens, хоча користувач їх не бачить. Модель генерує чернетку міркувань за будь-якого запиту, навіть простого.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Однозначного правила немає: більшість моделей навчені надавати перевагу system, але на практиці результат залежить від формулювань, довжини та конкретної моделі. Тому критичні вимоги краще перевіряти кодом, а не покладатися на одну інструкцію.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі мають менше параметрів для запам'ятовування рідкісних мовних патернів, а українських текстів у навчанні значно менше, ніж англійських. Через це зростає частка калькувань, русизмів і мовних збоїв, що видно в спотворених словах llama3.2:3b.

Висновок за завданням 2

System-повідомлення помітно змінює поведінку: для qwen3:4b вимога «українською, максимум двома реченнями» скоротила completion_tokens з 2574 до 441, а видиму відповідь — з 372 до 156 символів; у llama3.2:3b completion_tokens зменшились з 87 до 72. Проте надійність

обмежена: інструкція не гарантує якість змісту (llama виконала форму, але відповіла безглуздо) і не прибирає прихованих токенів міркування. Якщо модель ігнорує інструкцію, варто програмно перевіряти формат (кількість речень, мову), повторювати запит, знижувати temperature або змінити модель.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Варіант 7: «Поясни різницю між процесом і потоком. Коротко.» Скрипти providers.py та compare.py, дані: compare_results.json.

Таблиця 5. Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	2.97	1.65
completion_tokens	97	237
Довжина відповіді, символів	292	723
Мова відповіді	українською, зі спотвореними словами («процеср», «кількових»)	українською
Фактичних помилок	так: потік і процес описано як «масштабний процес… фізичних величин»	ні

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У compare.py — жодного. У providers.py змінюються лише значення трьох полів словника «cloud»: base_url, api_key (береться з .env) і model.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад.

Відповідь: Хмарна: 723 символи проти 292. Вона пояснила через аналогію (процес — окрема кухня, потоки — кухарі, що ділять плиту) і зазначила, що потоки одного процесу діляться спільною пам'яттю. Локальна назвала потік «масштабним процесом, який з'ясований за допомогою кількох фізичних величин», що не має стосунку до теми.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, обидва види: зміст хибний, а в тексті є спотворені слова («процеср», «кількових», «з'ясований»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого?

Відповідь: Хмарна: 1,65 с, локальна: 2,97 с. Ймовірно, серверне обладнання генерує токени швидше й перекриває мережеву затримку. До того ж локальна відповідь у 97 токенів при ~133 ток/с потребувала б лише ~0,7 с, а решту, схоже, займає фіксована затримка близько 2 с, яку видно й у benchmark.py (TTFT ~2,05 с). Її причину ми не досліджували.

Відповіді від чат-бота (Claude, Anthropic), крок 6

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема Ollama?

Відповідь: OpenAI першою масово запустила публічний API чат-моделей, тож навколо цього формату (/v1/chat/completions, ролі system/user/assistant) виросли бібліотеки й код. Іншим провайдерам і локальним серверам вигідніше було підтримати той самий формат, ніж змушувати розробників переписувати клієнтський код.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір — один із головних факторів, але не єдиний: значення мають якість і різноманітність навчальних даних, інструктивне донавчання, архітектура та мовне покриття. Велика модель зазвичай значно краща за малу на фактах і рідкісних мовах, але різниця залежить від завдання.

3. Чому хмарна модель відповіла швидше, попри затримку мережі?

Відповідь: Серверне обладнання (потужні прискорювачі, оптимізовані рушії інференсу, пакетна обробка) генерує токени швидше, ніж більшість локальних конфігурацій, і цей виграш перекриває мережеву затримку, особливо на довших відповідях. У цьому вимірюванні додатково впливає, ймовірно, фіксована затримка локального запуску.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Текст запитів залишає ваш комп'ютер, може зберігатися в логах провайдера й, залежно від умов, використовуватись для покращення моделей. У комерційних проєктах це зменшують угодами про обробку даних і режимами без збереження, маскуванням персональних даних, вибором регіону та, для найчутливіших даних, локальними чи приватними моделями.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — обмеження на кількість запитів і/або токенів за одиницю часу (а також на добу) для акаунта. Розробники працюють із ним через повтор із наростаючою затримкою, кешування, черги й пакетну обробку, розподіл навантаження між ключами чи провайдерами та перехід на платний тариф.

Висновок за завданням 3

Локальну модель я обрав би для чутливих даних, роботи без інтернету та простих задач, де помилка не критична: дані не залишають комп'ютер, вартість нульова. Хмарну — коли важливі точність і зв'язність відповіді: на простому питанні з інформатики локальна 3B-модель видала хибне пояснення, а хмарна відповіла правильно й повніше. Ціна хмари — залежність від мережі, ліміти тарифу та передавання тексту стороннім серверам. Перемикання між ними обійшлося зміною трьох значень у конфігурації.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Варіант 7: короткий промпт «Столиця Норвегії?», середній «Поясни в одному абзаці, що таке рекурсія.», довгий «Напиши інструкцію з 10 пунктів, як провести код-рев'ю.» benchmark.py: прогрів, три повтори з медіаною, без паралельних запитів, стрімінг. Дані: results.csv.

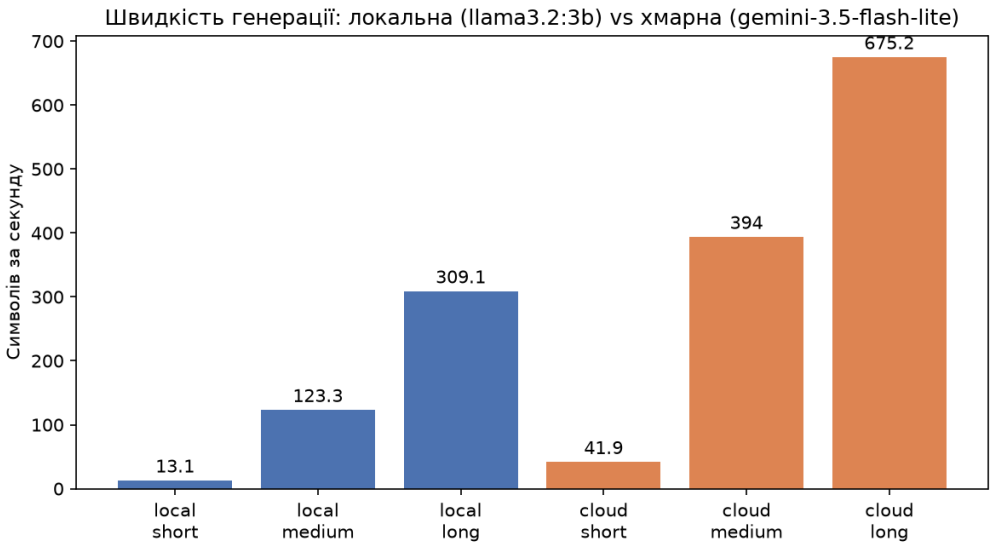
Таблиця 6. Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.051	2.298	13.1
локальна	llama3.2:3b	середній	2.082	2.823	123.3
локальна	llama3.2:3b	довгий	2.091	5.903	309.1
хмарна	gemini-3.5-flash-lite	короткий	0.634	0.669	41.9
хмарна	gemini-3.5-flash-lite	середній	0.576	1.167	394.0
хмарна	gemini-3.5-flash-lite	довгий	0.652	4.145	675.2

Таблиця 7. Холодний і теплий старт локальної моделі (llama3.2:3b, короткий промпт)

Показник	Холодний старт	Теплий старт
TTFT, с	3.73	2.043
Всього, с	3.815	2.134

Рисунок 1. Швидкість генерації у символах за секунду



Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: У хмарної: 0,58–0,65 с проти 2,05–2,09 с у локальної. Локальний TTFT майже не залежить від довжини промпту, що схоже на фіксовану затримку запуску на локальному сервері. Її точну причину ми не досліджували.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Відповідь: У наших даних збігається: хмарна швидша за обома показниками на всіх промптах (41,9 проти 13,1; 394 проти 123,3; 675,2 проти 309,1). Очікуваної розбіжності ми не спостерігали. На короткому промпті символи/с майже повністю визначаються TTFT.

3. Як змінилося співвідношення локальної та хмарної моделі від короткого промпту до довгого?

Відповідь: За швидкістю в символах/с хмарна випереджала локальну в 3,2 раза (короткий і середній) і в 2,2 раза (довгий); за загальним часом — у 3,4, 2,4 та 1,4 раза. Перевага хмари зменшується зі зростанням довжини відповіді, бо фіксована затримка локальної моделі розподіляється на довшу генерацію.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: У results.csv збережено лише медіани, самі повтори не фіксувались, тому оцінити їхній розкид за наявними даними не можна. Одного запуску недостатньо: він може потрапити на випадковий викид (фонове навантаження, мережа), тому медіана з трьох повторів виправдана.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту модель потрібно завантажити з диска в пам'ять GPU: TTFT 3.73 с проти 2.043 с, загальний час 3.815 с проти 2.134 с. У теплому стані ваги вже в пам'яті.

Відповіді від чат-бота (Claude, Anthropic), крок 7

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо?

Відповідь: Сервер приймає запит, токенізує промпт, обчислює його за один прохід (prefill) і генерує перший токен; для хмари додаються мережа й черга. Окремо вимірюють, бо це час, який користувач бачить як очікування до початку відповіді, а подальша швидкість визначає тривалість довгої відповіді.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?

Відповідь: У хмарі TTFT включає мережевий обмін, чергу й планування на сервері, а після початку потоку токени генеруються на потужних прискорювачах швидше, ніж на локальному

залізі. Тому хмара типово програє на коротких відповідях і виграє на довгих. У наших даних хмара виграла за обома показниками.

3. Чому перший запуск локальної моделі повільніший і що таке холодний старт?

Відповідь: Холодний старт — стан, коли модель ще не в пам'яті: сервер завантажує вагові коефіцієнти з диска й ініціалізує контекст, на що йдуть секунди. Далі модель лишається в пам'яті, тож наступні запити «теплі».

4. Чому беруть медіану кількох запусків, а не одне значення?

Відповідь: Окремі вимірювання зазнають випадкових стрибків через мережу, фонове навантаження й кеші. Медіана стійка до викидів і краще описує типову поведінку, ніж одиничне значення чи середнє.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Вхідні токени обробляються паралельно за один прохід (prefill), що добре завантажує GPU. Вихідні генеруються послідовно: кожен наступний залежить від попередніх, тому кожен потребує окремого проходу моделі.

Висновок за завданням 4

Для інтерактивного чат-бота важливіший TTFT: користувач бачить порожній екран, поки він триває, і саме ця величина визначає відчуття швидкості. Для фонової обробки великих обсягів тексту важливіша пропускна здатність (символи/с) і сумарний час, бо затримка першого токена розчиняється в тривалій роботі. У нашому вимірюванні хмарна модель виграла за обома показниками, але зі зростанням довжини відповіді її перевага зменшувалась.

Завдання 5. Оцінювання якості моделей за рубрикою

Варіант 7. Кожне завдання прогнано двічі для кожної моделі (quality.py, temperature=0,7), відповіді збережено у quality.json. Бали виставлено за рубрикою 0/1/2 окремо для кожного прогону; підсумок за завданням — середнє двох прогонів із округленням вниз (припущення автора). Завдання 2 перевірено через json.loads(): усі шість відповідей парсяться без правок.

Завдання: 1 — рік першого публічного кінопоказу братів Люм'єр (правильно: 1895, див. джерела); 2 — JSON event, year, city; 3 — стиснення тексту про перший кінопоказ до двох речень зі збереженням чисел (текст складено за відкритими даними: дата, місце, ціна квитка, кількість фільмів і дата патенту підтверджуються джерелами, тривалість фільмів і перелік стрічок наведено орієнтовно); 4 — функція top_rated(films, n); 5 — задача про зал на 180 місць (правильно: 6 сеансів, 1080 глядачів); 6 — чим документальне кіно відрізняється від художнього.

Таблиця 13. Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	0	2	2
2. Формат	2	2	2
3. Стиснення	0	1	2
4. Код	2	2	1
5. Логіка	0	1	1
6. Українська	0	1	2
Разом /12	4	9	10

Таблиця 14. Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	ні (0/0), але відповіді різні: 1882 і 1975	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	так (2/1)	ні
4. Код	ні	ні	так (2/0)

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
5. Логіка	ні	так (1/2)	так (1/2)
6. Українська	ні	ні	ні

Таблиця 15. Бали за прогонами (прогін 1 / прогін 2)

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	0 / 0	2 / 2	2 / 2
2. Формат	2 / 2	2 / 2	2 / 2
3. Стиснення	0 / 0	2 / 1	2 / 2
4. Код	2 / 2	2 / 2	2 / 0
5. Логіка	0 / 0	1 / 2	1 / 2
6. Українська	0 / 0	1 / 1	2 / 2

Пояснення спірних оцінок. qwen3:4b, завд. 3: прогін 2 відповів англійською (числа збережено), оцінено 1. qwen3:4b, завд. 5: у прогоні 1 не пораховано глядачів (1), у прогоні 2 вірно: 6 сеансів і 1080 глядачів (2). Gemini, завд. 4: прогін 2 повернув код мовою Prolog замість Python (0). Gemini, завд. 5: у прогоні 1 відповідь вірна, але перевірочна вставка помилково стверджує, що 6-й сеанс закінчиться о 23:45 (1). llama3.2:3b, завд. 4: прогін 1 розраховує на кортежі (x[1]), прогін 2 — на словники (x['rating']); обидва працюють для своєї структури, але інтерфейс нестабільний. Завдання 2: у llama у прогоні 2 вигадано місто «Шарлотсбурж»; рубрика оцінює лише парсинг, тож бал 2.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких завданнях виникла основна різниця?

Відповідь: Gemini — 10/12, qwen3:4b — 9/12, llama3.2:3b — 4/12. Основна різниця: завдання 1 (llama вигадала роки), 3 (llama втратила числа), 5 (llama дала хибні розрахунки й обірвану відповідь) та 6 (якість української).

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: У qwen3:4b — завдання 3 і 5, у Gemini — 4 і 5. У llama оцінки збіглися, але лише тому, що обидва прогони були однаково невдалі (у завданні 1 самі відповіді різні: 1882 і 1975). Отже, навіть сильніші моделі не гарантують повторюваності результату.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три: усі шість відповідей розпарсились json.loads() без правок. Але в завданні 4 Gemini обгортала код у markdown-блок, хоча просили «лише код».

4. Чи були проблеми з українською мовою?

Відповідь: У llama3.2:3b — серйозні: спотворені слова («наступпумu», «процеср»), домішки російської, нідерландської й англійської. У qwen3:4b — кальки («фіктивні елементи», «фікційні»), зіпсований символ у тексті, відповіді англійською в завданні 3 (прогін 2) та англійське поле event у JSON. У Gemini проблем не виявлено.

Приклад провалу (крок 7)

Модель: gemini-3.5-flash-lite, завдання 4 («генерація коду»), прогін 2 з 2.

Повний текст запиту:

> *top_rated(films, n) — повертає n найкращих фільмів за рейтингом. Лише код.*

Повний текст відповіді:

```prolog
top_rated(Films, N, Result) :-
    msort(Films, Sorted),
    reverse(Sorted, Reversed),
    length(Result, N),
    append(Result, _, Reversed).
```

Правильна відповідь: функція мовою Python, наприклад: def top_rated(films, n): return sorted(films, key=lambda f: f['rating'], reverse=True)[:n]. (Інший прогін тієї ж моделі видав саме Python-код.)

Пояснення. Запит не називає мови програмування, а запис «top_rated(films, n)» має вигляд сигнатури предиката. Інструкція «лише код» прибрала будь-які пояснення, за якими можна було б помітити непорозуміння. При temperature=0,7 модель в одному прогоні вгадала Prolog. Навіть як код Prolog відповідь сумнівна: msort/2 сортує терми за стандартним порядком, а не за полем рейтингу, якщо структура даних не збігається з цим порядком випадково. Висновок: у великої хмарної моделі провал стався не через «нерозумність», а через неоднозначний запит і вгадування за відсутності уточнення; саме тому результат між двома прогонами розійшовся. Для агентів це означає: мову, формат і структуру даних потрібно задавати явно та перевіряти результат програмно.

Відповіді від чат-бота (Claude, Anthropic), крок 8

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Крім рубрик на невеликому наборі завдань, використовують стандартизовані бенчмарки (MMLU, GSM8K, HumanEval тощо), людське парне порівняння з Elo-рейтингами, автоматичне оцінювання іншою моделлю («LLM-as-judge») і тестування на власних реальних даних з регресійними наборами. Зазвичай їх комбінують, бо кожен метод має упередження.

2. Чому моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: У навчальних даних JSON і код майже завжди оформлені markdown-блоками, тому ця звичка сильна й може перемагати коротку інструкцію. Надійніші способи — структуровані виходи (JSON-режим, схеми), валідація з повторним запитом або постобробка. У ваших прогонах усі три моделі в завданні 2 обійшлися без markdown, але Gemini обгортала код у завданні 4.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Частка україномовних текстів у навчальному корпусі значно менша за англомовну, а мала кількість параметрів обмежує здатність засвоїти рідкісніші мовні патерни. Звідси калькування, русизми й мовні збої.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Генерація стохастична: при temperature більше нуля токени вибираються випадково з розподілу, і невеликі відхилення на початку накопичуються. Додатково дрібні відмінності обчислень на GPU й пакетування можуть давати різні результати навіть за однакових налаштувань. Тут це видно по qwen3:4b: у завданні 5 один прогін не порахував глядачів, а інший порахував.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — впевнена, але хибна чи вигадана відповідь. Модель навчена передбачати правдоподібне продовження тексту, а не звіряти факти, а в навчанні рідко трапляються приклади чесного «не знаю», тому за слабкого сигналу вона генерує найправдоподібніший варіант. Приклад: llama3.2:3b назвала роки 1882 і 1975 замість 1895.

Висновок за завданням 5

Оцінка за рубрикою виявилась корисною, але лише частково об'єктивною: завдання 1, 2 і 4 перевіряються майже механічно, а 3, 5 і 6 потребують судження (наприклад, як оцінити англомовну відповідь у завданні 3). Два прогони на завдання — мала вибірка, але вони вже показали нестабільність (qwen3:4b у завданні 5, Gemini у завданні 4). Найпоказовіші для порівняння моделей — завдання 1 (факт), 5 (логіка) і 6 (мова): саме там llama3.2:3b провалилась. Завдання 2 (формат) цього разу не розрізнило моделей, а завдання 4 показало, що нестабільність властива й хмарній моделі. Для україномовного тексту я обрав би Gemini (природна мова за обидва прогони); серед локальних — qwen3:4b, але з перевіркою кальок і

мовних збоїв.

Завдання 6. Керування випадковістю: temperature і top_p

Варіант 7. Творче завдання: «Назва застосунку обліку особистих витрат». Фактичне питання: «Рік першого кінопоказу братів Люм'єр» (правильна відповідь — 1895, джерела наведено в кінці). Модель: llama3.2:3b (локально). Скрипт sampling.py, усі відповіді збережено в sampling_results.json.

Таблиця 16. Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0/5 (усі «1998»)
temperature	0.7	5	—
temperature	1.5	5	0/5
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17. Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так (але відповідь хибна: «1998 рік»)

Спостереження. При temperature=0,0 і top_p=0,1 п'ять відповідей на творче завдання збіглися дослівно (той самий список «Personal Finance Manager», «Expense Tracker»…). При top_p=0,5 усі п'ять відрізняються, але це варіації одного шаблону; при temperature=1,5 з'являються зламані слова й вкраплення інших мов. На фактичному питанні temperature=0,0 дала стабільну, але неправильну відповідь (1998), а temperature=1,5 — нісенітницю; лише в одній відповіді серед хаосу трапилось «1895-96». Метрика «унікальних із 5» груба: вона не відрізняє дрібні варіації від справжнього розмаїття.

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: 0,0 (максимум 0,2) з top_p за замовчуванням: потрібна передбачуваність для стабільного парсингу. Застереження: наш експеримент показав, що temperature=0 дає стабільну, але не обов'язково правильну відповідь, тож температура не замінює перевірку результату.

2. Яке — генератору варіантів заголовків?

Відповідь: Близько 0,7–1,0. Уже при 0,7 п'ять запусків дали п'ять різних відповідей, а при 1,5 вони перетворюються на мішанину слів і мов.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature змінює ймовірності всіх токенів, тому при 1,5 «хвіст» отримує реальний шанс і з'являються зламані слова та чужі мови. top_p лише обрізає хвіст і не змінює співвідношення між залишеними кандидатами: при 0,1 відповідь збіглась із жадібною, при 0,5 лишились варіації одного шаблону.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав експеримент із 10 запусків?

Відповідь: У нашому тесті так: 10 запусків дали 1 унікальну відповідь. Проте гарантії немає через особливості обчислень із плаваючою комою та паралелізм, а наша стабільна відповідь була ще й хибною (1998).

5. Наведіть найбезглуздішу відповідь при temperature=1,5 і поясніть, чому вона з'явилась.

Відповідь: Фактичне питання, temperature=1,5: «У роки перших фільгів у парне Квібуа и Дюши з'являються Квібан і Бенчак, хоча вони відсутні лише на пізніших кадрах.» Висока температура вирівнює розподіл, і малоймовірні токени (вигадані імена, фрагменти інших мов) отримують реальний шанс; кожен із них зсуває контекст, і подальші токени узгоджуються вже з нісенітницею.

Відповіді від чат-бота (Claude, Anthropic), крок 8

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Він ділить логіти z на T перед softmax: p_i = exp(z_i/T) / сума exp(z_j/T). При T менше 1 розподіл загострюється, при T більше 1 вирівнюється, а при T, що прямує до 0, вибір стає жадібним (argmax).

2. Чим top_p відрізняється від temperature — за яким механізмом?

Відповідь: top_p (nucleus sampling) сортує токени за ймовірністю й залишає найменший набір із сумарною ймовірністю не менше p, решту відкидає. Temperature змінює форму всього розподілу, не прибираючи кандидатів; top_p змінює склад кандидатів.

3. Чи є сенс змінювати обидва параметри одночасно?

Відповідь: Можна, але ефекти накладаються й їх важко інтерпретувати, тому зазвичай змінюють один параметр, а інший залишають за замовчуванням. Для порівняння налаштувань коректніше змінювати по одному за експеримент.

4. Чому навіть при temperature=0 модель не гарантує абсолютно однакову відповідь?

Відповідь: Через недетермінізм обчислень із плаваючою комою (порядок додавання при паралельних операціях на GPU), пакетування запитів на сервері та відмінності версій і апаратури. Для токенів із близькими ймовірностями мікрозміна чисел може змінити вибір.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для творчого тексту?

Відповідь: Для витягу структурованих даних, класифікації та фактичних відповідей зазвичай 0–0,3; для творчого тексту й брейнштормінгу 0,7–1,0, іноді вище. Значення 1,5 і вище зазвичай дає дедалі менш зв'язний текст.

Висновок за завданням 6

temperature і top_p регулюють випадковість вибору токена, а не «розумність» моделі: temperature змінює форму всього розподілу, top_p відсікає малоймовірний хвіст. На творчих завданнях підвищення випадковості дає розмаїття (з 1 до 5 унікальних відповідей), на фактичних — шкодить, а temperature=0 лише робить помилку відтворюваною (стабільне «1998»). Для наступних лабораторних, де модель повертає структуровані дані, я оберу temperature 0–0,2 і задаватиму її явно, бо значення за замовчуванням зазвичай ненульове (в Ollama близько 0,8) і може ламати стабільність формату.

Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (gemini-3.5-flash-lite)
Швидкість генерації	~133.0 ток/с (GPU); 13–309 симв./с залежно від промпту	~114.5 ток/с, але з тисячами токенів міркування	42–675 симв./с залежно від промпту
TTFT	2,05–2,09 с (холодний старт 3,73 с)	не заміряно	0,58–0,65 с
Якість, балів /12	4	9	10
Стабільність формату (завд. 2)	JSON валідний 2/2, але в прогоні 2 вигадано місто	JSON валідний 2/2, поле event англійською	JSON валідний 2/2; у завд. 4 — markdown і Prolog в одному прогоні
Потрібні ресурси	2,0 ГБ на диску; працює на RTX 3070	2,5 ГБ на диску; працює на RTX 3070	немає локальних вимог; інтернет і API-ключ

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (gemini-3.5-flash-lite)
Вартість	безкоштовно (електрика)	безкоштовно (електрика)	безкоштовний тариф з лімітами, далі платно
Куди йдуть дані	залишаються на комп'ютері	залишаються на комп'ютері	надсилаються на сервери Google
Працює без інтернету	так	так	ні
Коли обирати саме її	прості задачі та приватні дані, де помилка не критична, а затримка першого токена прийнятна	локально, коли потрібна краща логіка й мова, ніж у 3B, і можна платити часом за міркування	коли важливі точність, мова й швидкість, а дані не чутливі

Загальні висновки

• Локальна й хмарна моделі мають єдиний інтерфейс, тому перехід між ними зводиться до зміни конфігурації, а не логіки програми.

• Затримка складається з TTFT і швидкості генерації; висновок «яка модель швидша» без розділення цих показників некоректний.

• Режим міркування окупається на багатокрокових задачах і є перевитратою на простих запитах.

• System-повідомлення й промпт — ненадійний спосіб керувати форматом: пряма інструкція не гарантує ні форму, ні зміст, тож результат потрібно перевіряти програмно.

• Моделі недетерміновані, тому будь-яке вимірювання потребує повторів; стабільність відповіді (temperature=0) не означає її правильності.

Джерела

• Lumière brothers (Wikipedia): https://en.wikipedia.org/wiki/Lumi%C3%A8re_brothers — перший платний публічний кінопоказ 28 грудня 1895 року, Salon Indien du Grand Café, Париж; патент 13 лютого 1895 року.

• The Lumière Cinématographe (Google Arts & Culture, La Cinémathèque française): https://artsandculture.google.com/story/the-lumi%C3%A8re-cin%C3%A9matographe-la-cin%C3%A9math%C3%A8que-fran%C3%A7aise/zgWBKU2_7WzRLA

• 125 years ago, the first public cinema screening in France was held (Malay Mail, 29.12.2020): https://www.malaymail.com/news/showbiz/2020/12/29/125-years-ago-the-first-public-cinema-screening-in-france-was-held/1935643 — програма з десяти фільмів, квиток 1 франк.